



I LOCK IT Sharing

Cloud-API-Documentation V1.18

Contact:

Haveltec GmbH

Markus Weintraut

Friedrich-Franz-Str. 19

14770 Brandenburg an der Havel

Germany

markus@ilockit.bike

Index

1. Changelog	4
2. Introduction	7
3. Get a list of devices	8
4. Get a list of positions	10
5. Get a list of Events	12
6. Get command response	14
7. Get a list of available commands	15
8. Send command	17
9. Command description	19
10. Lock status events	30
11. Error events	31
12. Get firmware info	32
13. Start firmware update	33
14. Forwarding of Events and Positions	34

Disclaimer

The information contained in these documents is confidential, privileged and only for the information of the intended recipient and may not be used, published or redistributed without the prior written consent of haveltec GmbH.

The opinions expressed are in good faith and while every care has been taken in preparing these documents, haveltec makes no representations and gives no warranties of whatever nature in respect of these documents, including but not limited to the accuracy or completeness of any information, facts and/or opinions contained therein.

1. Changelog

Cloud API v1.18:

- Updated Chapter 9.8: Added locking without challenge response.

Cloud API v1.17:

- Updated Chapter 9.13: Added requirements for GSM firmware update.

Cloud API v1.16:

- Updated Chapter 9.8: Open/Close lock with automatic challenge response.

Cloud API v1.15:

- Updated Chapter 3: Added description of optional device attributes.

Cloud API v1.14:

- Updated Chapter 4: Storage period added.
- Updated Chapter 5: Storage period added.
- Updated Chapter 14: Changed server IP address.

Cloud API v1.13:

- Updated Chapter 5: Receive defined number of most recent events.

Cloud API v1.12:

- Updated Chapter 14: Define up to four forwarding endpoints.

Cloud API v1.11:

- Updated Chapter 9: Extended lock info with alarm and sound settings.

Cloud API v1.10:

- New Chapter 14:
 - o Forwarding of Events and Positions.

Cloud API v1.9:

- New Chapter 6:
 - o Get command response.

- Updated Chapter 8:
 - Edited response example.
- Updated Chapter 9:
 - Edited *mcFirmware* in response example.

Cloud API v1.8:

- Updated Chapter 5:
 - New event for wrong payload.
 - Event id in command response.
- Updated Chapter 9:
 - Added description for CAN info packet (12).
- Updated Chapter 13:
 - Response message extended.

Cloud API v1.7:

- Updated: Chapter 10:
 - Added status codes for gsm update result.
- Updated: Chapter 13:
 - Changed response code to 200.

Cloud API v1.6:

- Updated: Chapter 9:
 - Configuring proximity unlock feature.
- Updated: Chapter 9:
 - Response event for lock info command.
- Updated: Chapter 5:
 - Added new event types.

Cloud API v1.5:

- Updated: Chapter 8:
 - Commands have a time to live.
- Updated: Chapter 6:
 - Added time to live in response example.
- New: Chapter 12:
 - Get firmware information.

- New: Chapter 13:
 - o Start firmware update via GSM.

Cloud API v1.4:

- Updated: Chapter 9:
 - o New command for manual position update (11).
- Updated: Chapter 5:
 - o Get events for all devices within a group.
- Updated: Chapter 10:
 - o Get status about actual tracking mode.

Cloud API v1.3:

- Updated: Chapter 2:
 - o Hint about commands and different hardware versions.
- Updated: Chapter 5:
 - o Available events for different hardware versions.
- New: Chapter 6:
 - o How to get a list of available commands.
- New: Chapter 8:
 - o How to send a command to a device.
- New: Chapter 9
 - o Description of commands.
- New: Chapter 10
 - o Description of status events.
- New: Chapter 11
 - o Description of error events.

2. Introduction

This document describes the Rest protocol used to access the I LOCK IT cloud server. All I LOCK IT GPS devices send their location and status information to this cloud server.

With the following description it is possible to access the I LOCK IT cloud server from an external system and receive that information for the devices.

In order to receive the information via the API it is necessary to first create an user account at <https://tracking.ilockit.bike> we will then assign your account to your devices. It is currently not possible to assign yourself to your devices.

The API calls and commands, which are described in chapter 6, 8, 9, 10 and 11 are only effective from hardware version 4. This hardware version has a data connection to the GPS module and is able to communicate with the server (from firmware version 29).

3. Get a list of devices

This API call will return a list of devices which are assigned to your user account. When submitting an optional ID only the information for a certain device is returned. The result contains all relevant information of a lock. Depending on the production date, the attributes contain optional information, like color code, token or order number. The firmware version is available if the lock sends an info packet for the first time. The tracking mode is only available in S2 locks.

Authentication: basic authentication (e-mail address and password)

Method: Get

Endpoint: <https://tracking.ilockit.bike/api/devices>

Query Parameters: id (optional)

Body: none

HTTP Header: "accept: application/json"

Responses:

200 OK

400 Bad Request

401 Unauthorized

Response Example:

```
[
  {
    "id": 394,
    "attributes": {
      "colorCode": "BGRBRR",
      "token": "1B4D7A6999524B7AEAEF10FA36F330B3",
      "firmwareVersion": 34.9,
      "trackingMode": "lowPower",
      "orderNr": "BB01234",
    },
    "groupId": 9,
    "name": "ISHAREIT-2402216",
    "uniqueId": null,
    "appId": null,
    "status": "offline",
    "lastUpdate": "2022-01-17T16:37:11.000+0000",
  }
]
```



```
    "positionId": 58709,  
    "geofenceIds": null,  
    "phone": null,  
    "model": null,  
    "contact": null,  
    "category": null,  
    "disabled": false,  
    "configured": 3,  
    "expiryDate": null,  
    "uartAvailable": true,  
    "subscribed": false,  
    "subscriptionDate": null  
  }  
]
```

4. Get a list of positions

This API call returns a list of the positions for a certain device within a specified time. When no parameters are set a list of the last position of all devices is returned. Positions are available for the last 14 days at most.

Authentication: basic authentication (e-mail address and password)

Method: Get

Endpoint: <https://tracking.ilockit.bike/api/positions>

Query Parameters:

Key	Value
from	2020-04-05T00:00:00Z
to	2020-04-22T23:59:59Z
deviceId	64443

Body: none

HTTP Header: "accept: application/json"

Responses:

200 OK
400 Bad Request
401 Unauthorized

Response Example:

```
[
  {
    "id": 45368,
    "attributes": {
      "sat": 10,
      "distance": 16.16,
      "totalDistance": 76180.32,
      "motion": true
    },
    "deviceId": 64443,
    "type": null,
    "protocol": "topin",
    "serverTime": "2020-04-23T13:04:32.000+0000",
    "deviceTime": "2020-04-23T13:04:30.000+0000",
    "fixTime": "2020-04-23T13:04:30.000+0000",
    "outdated": false,
```

```
"valid": true,  
"latitude": 52.41245833333333,  
"longitude": 12.554406666666667,  
"altitude": 0,  
"speed": 1.07991,  
"course": 84,  
"address": null,  
"accuracy": 0,  
"network": null  
}  
]
```

5. Get a list of Events

This API call returns a certain number of the most recent events of all devices in a group. The desired number of events must be specified with the parameter *limit*, which must not be greater than 10.000. Events are available for the last 14 days at most.

When using an older I SHARE IT lock (below hardware version 4) the event *deviceOnline* also signals, that an Alarm has been triggered at the lock. This does not apply for newer I SHARE IT locks. These locks will trigger the *deviceOnline* event every time the module connects to the server. In case of a theft attempt, these locks will trigger an *alarm* event.

These locks can also send much more different events:

- *deviceOnline* Device gets online
- *deviceUnknown* Device gets offline
- *lockInfo* Information about battery state of the lock
- *lockStatus* Status codes of the lock
- *lockingChallenge* Challenge for un-/locking command
- *error* Error codes, which gives information about invalid commands
- *acknowledge* ACK for valid commands
- *unknown* Indicates an unknown command received from I LOCK IT
- *requestFwPacket* Event for tracking firmware update process

If the user sends a command, commits the wrong payload or a previously sent command expires, it will also be logged as event. Therefore, the following event types are used:

- *command* Commands sent via API are logged as event
- *commandExpired* Expired command from queue
- *wrongPayload* Command was supplied with incorrect data

If the command requests data from the lock (valid from firmware version 34.0) the generated event will contain the event id of the request command. This enables the user to keep track of the requested data.

Authentication: basic authentication (e-mail address and password)

Method: Get

Endpoint: <https://tracking.ilockit.bike/api/reports/events>

Query Parameters: *groupId* and *limit* must be passed

Key	Value
limit	5000
groupId	4

Body: none

HTTP Header: "accept: application/json"

Responses:

200 OK

400 Bad Request

401 Unauthorized

Response Example:

```
[
  {
    "id": 20912,
    "attributes": {},
    "deviceId": 64443,
    "type": "deviceOnline",
    "serverTime": "2020-04-20T10:04:54.000+0000",
    "positionId": 0,
    "geofenceId": 0,
    "maintenanceId": 0,
    "commandId": 0
  }
]
```

6. Get command response

This API call can only be used with I LOCK ITs which run on firmware version 34.0 and newer. It returns a single event, which is a response to a prior sent command. As seen in the response example in chapter 8, you get an *eventId*, when sending a command. This ID can be used to get the corresponding answer of the lock, e.g. acknowledge, locking challenge, etc.

Authentication: basic authentication (e-mail address and password)

Method: Get

Endpoint: <https://tracking.ilockit.bike/api/events>

Query Parameters:

Key	Value
eventId	82142

Body: none

HTTP Header: "accept: application/json"

Responses:

200 OK

400 Bad Request

401 Unauthorized

Response Example:

```
{
  "id": 82143,
  "attributes": {
    "eventId": 82142,
    "command": "signalSound"
  },
  "deviceId": 358,
  "type": "acknowledge",
  "serverTime": "2021-09-09T07:25:14.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

7. Get a list of available commands

This API call will return a list of commands which are assigned to your account. When submitting an optional deviceId the available commands for a certain device are returned. Every command itself has also an ID, e.g command signalSound has ID 4. These IDs must be used, when sending commands to the device (see chapter 8). The time to live (ttl) of the command is specified in minutes.

Authentication: basic authentication (e-mail address and password)

Method: Get

Endpoint: <https://tracking.ilockit.bike/api/commands>

Query Parameters: deviceId (optional)

Body: none

HTTP Header: "accept: application/json"

Responses:

200 OK

400 Bad Request

401 Unauthorized

Response Example:

```
[
  {
    "id": 3,
    "attributes": {},
    "deviceId": 0,
    "type": "locking",
    "textChannel": false,
    "description": "Opens or closes an I LOCK IT",
    "ttl": 1,
  },
  {
    "id": 4,
    "attributes": {},
    "deviceId": 0,
    "type": "signalSound",
    "textChannel": false,
    "description": "Activates a signal sound at I LOCK IT",
    "ttl": 3,
  }
]
```

```
    },  
    {  
      "id": 5,  
      "attributes": {},  
      "deviceId": 0,  
      "type": "getLockInfo",  
      "textChannel": false,  
      "description": "Requests information of an I LOCK IT",  
      "ttl": 5,  
    }  
  ]
```


8. Send command

This API call will send a command to one of your I LOCK IT. You need to specify the receiving device with the appropriate `deviceId` and the command with an `id` (see chapter 6). The attributes are optional and not needed for all commands. To send a command, you first have to check if the command is available for the specified device. If the device is online and the command was sent successfully, the API call will return 200. If the device is currently offline the command is queued and sent if the device gets online. The API call will then return status code 202.

Every command has a fixed “time-to-live” after which the command is removed from the queue if the target device is offline. The TTL can be seen in the list of available commands (see chapter 6). Furthermore, every sent command has an *eventId*, which can be used to find the corresponding response from I LOCK IT (see chapter 6).

If a command is transferred successfully, the device sends either an acknowledge or the enquired data to the server. On the server-side the results are saved as events (see chapter 5).

Authentication: basic authentication (e-mail address and password)

Method: Post

Endpoint: <https://tracking.ilockit.bike/api/commands/send>

Query Parameters: none

Body: `deviceId` (mandatory), `id` (ID of command, mandatory), `attributes` (optional) in JSON format
Example:

```
{
  "deviceId": 274,
  "id": 4,
}
```

HTTP Header: “accept: application/json”

Responses:

200 OK

202 Accepted

400 Bad Request

401 Unauthorized

Response Example:

```
{  
  "id": 4,  
  "attributes": {},  
  "deviceId": 274,  
  "type": null,  
  "textChannel": false,  
  "description": null,  
  "ttl": 0,  
  "eventId": 82165,  
}
```

9. Command description

This chapter describes all commands, which are available on the server. Note, that maybe not all commands are available for your I LOCK IT and your user account. The IDs of the commands must be detected by the API call described in chapter 6.

1. Get lock info

- Description: Requests the firmware version, current battery (0-100), lock state (0-2), charging status (0-1), theftMode (0-1), alarmArmed (0-1), alarmSettings and soundSettings (see I LOCK IT BLE API for description of the available values).

- Example API call:

```
{  
  "deviceId": 274,  
  "id": 5  
}
```

- Response event:

```
{  
  "id": 25719,  
  "attributes": {  
    "firmwareVersion": 29,  
    "batteryLevel": 100,  
    "lockState": 2,  
    "charging": 1,  
    "theftMode": 0,  
    "alarmArmed": 1,  
    "alarmSettings": 16,  
    "soundSettings": 6,  
  },  
  "deviceId": 274,  
  "type": "lockInfo",  
  "serverTime": "2020-12-16T07:55:36.000+0000",  
  "positionId": 0,  
  "geofenceId": 0,  
  "maintenanceId": 0,  
  "commandId": 0  
}
```

2. Start signal sound

- Description: In order to ease the search of an I LOCK IT, you can remotely start a signal sound which will be emitted by the lock.

- Example API call:

```
{  
  "deviceId": 274,  
  "id": 4  
}
```

```

    }
-   Response event:
    {
      "id": 25721,
      "attributes": {
        "command": "signalSound"
      },
      "deviceId": 274,
      "type": "acknowledge",
      "serverTime": "2020-12-16T08:13:32.000+0000",
      "positionId": 0,
      "geofenceId": 0,
      "maintenanceId": 0,
      "commandId": 0
    }

```

3. Arm alarm

- Description: Arms the alarm functionality of the lock. An alarm will be set off if the locked bike is moved.
- Example API call:

```

{
  "deviceId": 274,
  "id": 6
}

```

```

-   Response event:
    {
      "id": 25736,
      "attributes": {
        "command": "configuration",
        "setting": "alarm",
      },
      "deviceId": 274,
      "type": "acknowledge",
      "serverTime": "2020-12-17T16:18:24.000+0000",
      "positionId": 0,
      "geofenceId": 0,
      "maintenanceId": 0,
      "commandId": 0
    }

```

4. Disarm alarm

- Description: Disarm the alarm functionality of the lock. No more alarms will be set off when the bike is being moved.

- Example API call:

```
{  
  "deviceId": 274,  
  "id": 7  
}
```

- Response event:

```
{  
  "id": 25737,  
  "attributes": {  
    "command": "configuration",  
    "setting": "alarm",  
  },  
  "deviceId": 274,  
  "type": "acknowledge",  
  "serverTime": "2020-12-17T16:18:34.000+0000",  
  "positionId": 0,  
  "geofenceId": 0,  
  "maintenanceId": 0,  
  "commandId": 0  
}
```

5. Turn off an active alarm

- Description: Deactivates an active alarm of the lock. The alarm functionality will stay armed and it sets off again if the bike is moved again.
- Example API call:

```
{  
  "deviceId": 274,  
  "id": 12  
}
```

- Response event:

```
{  
  "id": 25769,  
  "attributes": {  
    "command": "alarmOff"  
  },  
  "deviceId": 274,  
  "type": "acknowledge",  
  "serverTime": "2020-12-16T15:58:34.000+0000",  
  "positionId": 0,  
  "geofenceId": 0,  
  "maintenanceId": 0,  
  "commandId": 0  
}
```

6. Restart device

- Description: Restarts the lock.
- Example API call:

```
{  
  "deviceId": 274,  
  "id": 8  
}
```

- Response event:

```
{  
  "id": 25728,  
  "attributes": {  
    "command": "rebootDevice"  
  },  
  "deviceId": 274,  
  "type": "acknowledge",  
  "serverTime": "2020-12-16T15:58:34.000+0000",  
  "positionId": 0,  
  "geofenceId": 0,  
  "maintenanceId": 0,  
  "commandId": 0  
}
```

7. Restart GPS

- Description: Restarts the GPS module of the lock.
- Example API call:

```
{  
  "deviceId": 274,  
  "id": 11  
}
```

- Response event:

```
{  
  "id": 25745,  
  "attributes": {  
    "command": "gpsReset"  
  },  
  "deviceId": 274,  
  "type": "acknowledge",  
  "serverTime": "2020-12-16T15:58:34.000+0000",  
  "positionId": 0,  
  "geofenceId": 0,  
  "maintenanceId": 0,  
  "commandId": 0  
}
```

8. Open/Close lock

- Description: Opens or closes the lock. Dependent on the firmware version there are different possibilities to send a locking command to the lock. From **firmware version 35.5** onwards the user can choose, if a challenge response must be completed before locking or if the command is executed immediately without challenge response.

The challenge response needs two steps to achieve a higher layer of security. If the devices firmware is below version 34.0, the process must be realised by the user. If it's above, the user can do a simpler request and the backend will do the authentication. It's based on the authentication process via BLE (see I LOCK IT BLE API document).

- **Firmware below version 34.0:**

In the first step the user sends a 16-byte long seed to the lock (locking request). The lock sends a 16-byte long random number which is encrypted with the user key, back to the server. The user has to decrypt the number, modify the last byte and encrypt it with its own user key again. The second API call contains the modified result and the locking direction "open" or "close" (locking response).

- Example API calls:

- o Locking request:

```
{
  "deviceId": 274,
  "id": 3,
  "attributes": {
    "lockingSeed": "2c12cb4ad877d738ba3661877540d483"
  }
}
```

- o Locking response:

```
{
  "deviceId": 274,
  "id": 3,
  "attributes": {
    "direction": "open",
    "lockingResp": "2c12cb4ad877d738ba3661877540d483"
  }
}
```

- Response events:

- Locking request:

```
{
  "id": 25750,
  "attributes": {
    "challenge": "4BA318B9B079C759B2DE2815D4F690EC"
  },
  "deviceId": 274,
  "type": "lockingChallenge",
  "serverTime": "2020-12-16T14:31:11.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

- Locking response:

```
{
  "id": 25751,
  "attributes": {
    "command": "locking"
  },
  "deviceId": 274,
  "type": "acknowledge",
  "serverTime": "2020-12-16T14:32:16.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

- **Firmware from version 34.0:**

In this case, the user just sends a 16-byte long random seed and the desired locking direction to the lock. The backend will then do the challenge-response procedure with the lock. At the end the lock will send the acknowledge for the locking request. Nevertheless, it is still possible to use the old method and do the process on your own.

- Example API call:

- Locking request:

```
{
  "deviceId": 274,
  "id": 3,
  "attributes": {
    "lockingSeed": "2c12cb4ad877d738ba3661877540d483",
    "direction": "open"
  }
}
```

- Response events:

- Locking response:

```
{
  "id": 25751,
  "attributes": {
    "command": "locking"
  },
  "deviceId": 274,
  "type": "acknowledge",
  "serverTime": "2020-12-16T14:32:16.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

- **Firmware from version 35.5:**

In this case the user can choose one of the above options or can send the locking command without challenge response. This is a simpler approach and can help solve problems such as poor network reception because the needed data required for transfer is much less.

- Example API call:

- Locking request:

```
{
  "deviceId": 443,
  "id": 3,
  "attributes": {
    "direction": "open"
  }
}
```

- Response event:

- Locking response:

```
{
  "id": 4438,
  "attributes": {
    "eventId": 4437,
    "command": "locking"
  },
  "deviceId": 443,
  "type": "acknowledge",
  "serverTime": "2023-11-07T13:36:00.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

9. Send theft message

- Description: If a bike gets stolen, you can notify the lock about it to change its location update frequency. To activate the theft mode, you have to add the attribute key "theft" and the value "true". To deactivate the mode set the value to "false"

- Example API call:

```
{
  "deviceId": 274,
  "id": 10,
  "attributes": {
    "theft": "true"
  }
}
```

- Response event:

```
{
  "id": 25701,
  "attributes": {
    "command": "theftMessage"
  },
  "deviceId": 274,
  "type": "acknowledge",
  "serverTime": "2020-12-16T08:18:52.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

10. Configure I LOCK IT

- Description: This allows you to configure an I LOCK IT. For configuration of alarm and sound settings, you can do this in the same way as over BLE (see I LOCK IT BLE API for description of the available values). The proximity unlock feature can be activated/deactivated with true/false.

- Example API call:

- o Configure alarm mode:

```
{
  "deviceId": 274,
  "id": 9,
  "attributes": {
    "alarmmode": 34
  }
}
```

```
}  
}
```

- Configure sound setting:

```
{  
  "deviceId": 274,  
  "id":9,  
  "attributes":  
  {  
    "sound":5  
  }  
}
```

- Configure lock setting:

```
{  
  "deviceId":274,  
  "id":9,  
  "attributes":  
  {  
    "lockConfig":  
    {  
      "proximityUnlock":true  
    }  
  }  
}
```

- Response events:

- Configure alarm mode:

```
{  
  "id": 25956,  
  "attributes": {  
    "command": "configuration",  
    "setting": "alarmmode"  
  },  
  "deviceId": 274,  
  "type": "acknowledge",  
  "serverTime": "2020-12-12T18:18:52.000+0000",  
  "positionId": 0,  
  "geofenceId": 0,  
  "maintenanceId": 0,  
  "commandId": 0  
}
```

- Configure sound setting:

```

{
  "id": 25957,
  "attributes": {
    "command": "configuration",
    "setting": "sound"
  },
  "deviceId": 274,
  "type": "acknowledge",
  "serverTime": "2020-12-12T18:19:52.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}

```

- Configure lock setting:


```

{
  "id": 25932,
  "attributes": {
    "command": "configuration",
    "setting": "lockConfig"
  },
  "deviceId": 274,
  "type": "acknowledge",
  "serverTime": "2020-12-12T18:19:52.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}

```

11. Request position

- Description: This allows you to query a location update of a lock. This works only, if the GPS module gets a fix for its location. In this case there is no event, but a new position entry created on the server. If the module can't get a location fix, neither a position nor an event will be created.
- Example API call:

```

{
  "deviceId": 274,
  "id": 15
}

```

12. CAN info event

- Description: If there's an additional CAN module installed in the lock, it can read data from a connected peer. The CAN info packet is automatically sent, if the lock is opened/closed or every 30 minutes together with the lock status message. At the moment only keys beginning with "mc" are used.

- Response event:

```
{
  "id": 32691,
  "attributes": {
    "bmsBatterySoc": 0,
    "bmsRemainingCapacity": 0,
    "bmsFirmware": 0,
    "bmsChargeCounter": 0,
    "bmsErrorCodes": 0,
    "bmsWarnCodes": 0,
    "consoleFirmware": 0,
    "mcFirmwareIndex": 0,
    "mcFirmware": "DA15051807",
    "mcMileageTravel": 569,
    "mcMileageTotal": 569,
    "mcMileageRemaining": 156,
    "mcBatteryLevel": 26,
    "mcCharging": 0,
    "mcSystemInfo": 0,
    "eventId": 0
  },
  "deviceId": 358,
  "type": "canInfo",
  "serverTime": "2021-08-10T11:04:16.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

10. Lock status events

The lock also sends status messages to the server by itself. The event has the type *lockStatus* and the *statusCode* gives details about the status event, as seen in this example:

```
{
  "id": 25752,
  "attributes": {
    "statusCode": "openedGsm"
  },
  "deviceId": 274,
  "type": "lockStatus",
  "serverTime": "2020-12-16T14:32:22.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

The lock sends these events in the following cases:

<u>statusCode</u>	<u>Description</u>
alarm	Alarm was triggered
openBlocked / closeBlocked	Locking bolt was blocked while opening / closing
lockMoved	Lock was moved while trying to lock
closedButton / closedApp / closedGsm	Lock was closed via button / app / gsm
openedCode / openedApp / openedGsm	Lock was opened via colorcode / app / gsm
trackAll*	Lock is in constant tracking mode
trackLpw*	Lock is in low power tracking mode
gsmUpdateSuccessful / gsmUpdateFailed	Result of GSM update

* Actual tracking mode is also saved in devices attributes (trackingMode: always/lowPower).

11. Error events

If there is something wrong with a received GSM command, the lock sends error messages to the server. The event has the type *error* and the *errorCode* gives details about the status event, as seen in this example:

```
{
  "id": 25758,
  "attributes": {
    "errorCode": "wrongLength"
  },
  "deviceId": 274,
  "type": "lockStatus",
  "serverTime": "2020-12-16T14:32:22.000+0000",
  "positionId": 0,
  "geofenceId": 0,
  "maintenanceId": 0,
  "commandId": 0
}
```

The lock sends these events in the following cases:

<u>errorCode</u>	<u>Description</u>
wrongCommand	Unknown command received
wrongCrc	CRC doesn't fit to received message
wrongLength	Wrong message size received
authFailed	Authentication failed (e.g. locking command)

12. Get firmware info

This API call returns information about the actual available firmware for a specific lock. The response contains its version and the URL to the firmware package. The request must contain the name of the lock (case sensitive) as query parameter. If there is no firmware available for this specific lock, the response contains an empty “firmware” key.

Authentication: none

Method: Get

Endpoint: <https://tracking.ilockit.bike/api/devices/firmware>

Query Parameters: name

Body: none

HTTP Header: “accept: application/json”

Responses:

200 OK

400 Bad Request

401 Unauthorized

Response Example:

```
{
  "firmware": {
    "version": "32.1",
    "url": "https://info.ilockit.bike/sharing/ota_ishareit_plus-gps_v32_1.zip"
  }
}
```


13. Start firmware update

This API call will start a firmware update via GSM for a specific lock. This is only possible on locks with bootloader version 2 and firmware version 35.2 or above. The update is only started, if a newer firmware is available for that device. The progress of the transferred packages can be observed with the lock events (see chapter 5).

Authentication: basic authentication (e-mail address and password)

Method: Post

Endpoint: <https://tracking.ilockit.bike/api/devices/firmware>

Query Parameters: deviceId

Body: none

HTTP Header: "accept: application/json"

Responses:

200 Success

400 Bad Request

401 Unauthorized

Response Example:

```
{
  "295": "Firmware update started",
  "333": "deviceId invalid",
  "313": "Firmware of device is up to date",
  "303": "Firmware file not found"
}
```

14. Forwarding of Events and Positions

For larger fleets it's not recommended to poll the needed information via the API, because this would lead to an unnecessary increase of server requests. Therefore, we've implemented an interface, where all events and positions of a device can be forwarded to external servers. It can be configured if just events will be forwarded or also the positions of the locks.

Devices are arranged in groups. Every customer has at least one group of devices. The forwarding options can be configured on a group level by any group member. The corresponding ID of a group can be found in the device data.

To prevent arbitrary insertion of fake data, the forwarding endpoint can be secured with standard HTTP basic authentication and it's recommended to set up an additional IP filter for the IP of our tracking server (192.71.151.108). Therefore, the endpoint has to be secured with a username and password. These credentials will be included in the header of every forwarded event/position. They can be configured with the appropriate header key and must be base64-encoded. In the example below the credentials are "user:password".

It's possible to configure up to four different endpoints. The configuration of the event forwarder can be done via an API call. In that call all desired endpoints must be defined, that means that previous configurations will be completely overwritten. If a specific endpoint is not defined, it will be removed from our server.

Authentication: basic authentication (e-mail address and password)

Method: Post

Endpoint: <https://tracking.ilockit.bike/api/groups/forwarder>

Query Parameters: none

Body: groupId (Integer, mandatory), devURL (String, optional), devHeader (String, optional), devForwardPosition (Boolean, optional), qaURL (String, optional), qaHeader (String, optional), qaForwardPosition (Boolean, optional), stageURL (String, optional), stageHeader (String, optional), stageForwardPosition (Boolean, optional), prodURL (String, optional), prodHeader (String, optional), prodForwardPosition (Boolean, optional) in JSON format

Example:

```
{
  "groupId": 27,
  "devURL": "https://your.domain.com/devEndpoint",
  "stageURL": "https://your.domain.com/stageEndpoint",
  "stageHeader": "Authorization: Basic dXNlcjpwYXNzd29yZA==",
  "stageForwardPosition": true,
```

```
    "prodURL": "https://your.prodDomain.com/endpoint",  
    "prodHeader": "Authorization: Basic dXNlcjpwYXNzd29yZA==",  
    "prodForwardPosition": true  
  }
```

HTTP Header: "accept: application/json"

Responses:

200 Success

400 Bad Request

401 Unauthorized